

Herramientas de Ingeniería: Selección, Aplicación y Adaptación en el Radar Acústico ESP32

Emanuel Rojas
Mariana González

CE1110 – Análisis de Señales Mixtas, Instituto Tecnológico de Costa Rica

Contexto

Este documento detalla, según lo solicitado en la especificación del proyecto, las herramientas de ingeniería usadas para construir el radar acústico basado en ESP32 y cómo se seleccionaron, aplicaron y adaptaron. Como hilo conductor se usa el problema complejo de ingeniería identificado durante las pruebas de hardware y documentado en el artículo científico: el sistema mide correctamente a 15 cm y 22 cm, pero a partir de allí no selecciona de forma confiable el candidato de distancia correcto entre las reflexiones del objeto y el ruido de fondo del entorno (*clutter*). Resolver esta ambigüedad requirió combinar herramientas de hardware, software y análisis de datos en varias etapas del proyecto.

1. Selección de técnicas, recursos y herramientas

La selección de cada herramienta respondió a una variable concreta del problema: procesar señales digitales en tiempo real con recursos limitados, y luego distinguir un eco real de reflexiones parásitas.

Hardware. Se eligió una placa ESP32 DevKit genérica porque integra ADC, temporizadores de hardware y suficiente RAM para arreglos de cientos de muestras en `double`, sin requerir un segundo microcontrolador dedicado. Como transductor de salida se usó un parlante pequeño excitado a través de un amplificador clase D PAM8403, elegido por su bajo costo y por aislar la señal digital del ESP32 de la potencia de audio. Como receptor se usó un micrófono electret, adecuado para el ancho de banda de 2–8 kHz del chirp transmitido.

Software de prototipado. Se seleccionó Python (con `numpy` solo para arreglos y `matplotlib` para graficar) para validar los algoritmos de DFT, FFT y correlación antes de tocar hardware, porque permite iterar y depurar en minutos lo que en el microcontrolador toma mucho más tiempo por ciclo de compilación/carga.

Software embebido. Se usó el Arduino IDE por su soporte directo al ESP32 y a interrupciones de temporizador de hardware, necesarias para generar el chirp con precisión de muestreo. Para el cálculo de la FFT en el microcontrolador se adoptó la biblioteca `arduinoFFT` en lugar de portar la FFT propia (validada en Python), por su optimización ya probada para el núcleo Xtensa y su menor huella de memoria.

Control de versiones y documentación. Se usó Git con flujo de ramas `develop/feature` para cumplir el requisito de avance incremental del curso, y LaTeX/Overleaf para el artículo científico y este documento, por su control de formato reproducible entre ambos integrantes.

Apoyo de inteligencia artificial. Se usó un asistente de IA (Claude) como base de apoyo puntual: no para escribir el firmware ni los scripts de Python del equipo, sino para consultarlo cuando el equipo se topó con un problema concreto o quería investigar algo más a fondo antes de decidir. El caso principal fue el problema de selección de eco descrito arriba: se le pidió analizar, sin modificar código, qué factores de hardware (potencia del parlante, ruido del PAM8403, ganancia del micrófono) y de software (heurística de selección de picos, lóbulos de autocorrelación del chirp, ausencia de filtrado) podían explicar la falla

más allá de 20 cm. También se usó para dar formato a la documentación técnica del proyecto.

2. Aplicación de las técnicas al problema

El flujo de trabajo fue el mismo para cada etapa del proyecto: primero validar en Python con señales simuladas y ruido controlado, luego portar al ESP32 y contrastar contra mediciones físicas reales.

Para el problema de selección de eco específicamente, el equipo primero reprodujo el chirp y capturó el eco a distancias conocidas (15, 20, 22 y 25 cm) usando el monitor serie del Arduino IDE como única herramienta de visualización disponible en esta etapa, ya que la pantalla LCD prevista en el diseño original aún no se había integrado. Los datos recolectados mostraron que el sistema medía correctamente hasta 20 cm pero elegía el candidato equivocado más allá de esa distancia.

Con esos datos, el equipo consultó al asistente de IA para acotar las hipótesis plausibles antes de decidir qué cambiar en el firmware. De ese análisis (hardware vs. software) se concluyó que la causa más probable era la heurística de selección de picos – tomar el pico de mayor magnitud en toda la ventana, en vez del primer pico por encima de un umbral –, ya que a mayor distancia el eco real se debilita mientras las reflexiones fijas del entorno mantienen magnitud casi constante. El equipo aplicó ese diagnóstico modificando directamente el código de detección de picos en `correlacion.ino`, y aplicó Git para versionar el cambio como una rama de trabajo independiente, siguiendo el flujo exigido por el curso.

3. Adaptación de las técnicas y herramientas

Ninguna herramienta se aplicó sin modificaciones respecto a su uso típico en la literatura de radar. La regla clásica de correlación cruzada como filtro adaptado –tomar el máximo global de la ventana de correlación– funcionó sin problema en las simulaciones de Python, donde no hay clutter físico. Al pasar a hardware real, el equipo adaptó esa regla a un criterio de *primer* pico local por encima de un umbral relativo tras un intervalo de guarda, para dos de los tres candidatos calculados; para el tercer candidato (diferencia contra una calibración base sin objeto) se conservó el criterio de máximo global, porque ahí el clutter estático ya fue restado y el pico remanente más fuerte sí debería corresponder al objeto nuevo.

La biblioteca `arduinoFFT`, pensada originalmente solo para análisis espectral, se reutilizó además como motor de correlación mediante FFT (multiplicación por el conjugado y transformada inversa), evitando mantener dos implementaciones de FFT distintas dentro de la memoria limitada del ESP32. El intervalo de guarda, que en la teoría de radar pulsado se fija para exceder la duración del pulso, se adaptó para derivarse directamente de la fórmula de rango mínimo del curso ($R_{\text{mín}} \approx v_s T_{\text{chirp}}/2$), en vez de fijarse de forma empírica.

El uso del asistente de IA también se adaptó durante el proyecto: al inicio se empleó principalmente para resolver dudas conceptuales de DSP (por ejemplo, la relación entre convolución y correlación en el dominio de la frecuencia); en la etapa de hardware se volvió una herramienta de diagnóstico dirigido – se le entregó el código real y los datos de mediciones, y se le pidió explicar posibles causas, dejando siempre en el equipo la decisión de qué implementar. Esta adaptación quedó documentada en el historial de commits del repositorio, que muestra los cambios aplicados por el equipo después de cada ronda de análisis.

Queda pendiente adaptar el mismo enfoque – prototipar en Python, diagnosticar con apoyo de IA, aplicar y versionar en el ESP32 – para resolver por completo la ambigüedad de los tres candidatos de distancia y validar el sistema en todo el rango de diseño, tal como se plantea en las recomendaciones del artículo científico.